LUMO

# 流明 · 设计文档

PDF 扫描增强 · 文本识别 · 智能压缩

**版本** 0.1.0

**平台** macOS 13+

**形态** 原生 App（SwiftUI）+ 命令行工具 lumo-cli

**依赖** 零第三方依赖：仅 CoreGraphics / Core Image / Vision / ImageIO / zlib

本文档随 App 内置，可在「关于」页随时打开。

# 目录


01 产品定位与命名

02 三条设计支柱

03 视觉与交互规范

04 技术架构：为什么是零依赖

05 内存：处理能力不该被文件大小卡住

06 流水线：感知 → 推荐 → 增强 → 识别 → 压缩

07 增强模式：从「滤镜清单」到「效果选择」

08 压缩：自适应逐页路由与分层压缩（MRC）

09 关于 JBIG2 的诚实说明

10 已验证与已知限制

11 设计取舍记录：为什么是这些选项


## 01 · 产品定位与命名

**Lumo**（流明，光通量单位）解决的问题很具体：用 Adobe Acrobat 处理扫描 PDF， 要在「扫描增强 → 文本识别 → 压缩」三个面板之间来回切换、逐项调参， 调完不满意还要反复重来。整个过程里用户做的决定，大部分其实机器可以做对。

流明是「照在纸上有多亮」的单位。这个名字选定的产品气质是：**把纸面清理干净， 但不改变纸上的内容**。它不是图像编辑器，不是 OCR 引擎的壳，也不是压缩器的前端—— 它是一条自动工作流，用户只需要回答「要不要更小的文件」。

## 02 · 三条设计支柱

SENSE

### 感知

打开文件先做体检：有没有文字层、彩色还是灰阶、估计分辨率、 倾斜角、噪声强度、背景是否不匀。所有后续决定都建立在体检结果上， 而不是建立在用户的猜测上。

SMART

### 省心

给推荐而不是给参数表。已经识别过的文件自动跳过 OCR； 干净的文字页直接走单色极限压缩。用户可以改，但默认值必须是对的。

LIGHT

### 清新

多档压缩方案 + 用真实抽样页外推的体积预估。 「估计多大」必须是量出来的，不是拍脑袋的公式。

# 03 · 视觉与交互规范

## 色彩

| 令牌 | 值 | 用途 |
| --- | --- | --- |
| primary | #14B8A6 | 主操作、选中态、强调 |
| primaryDeep | #0D9488 | 数字、结论性文字 |
| primarySoft | #E6F7F4 | 选中卡片底色 |
| panelAlt | #F7FAFC | 次级面板 |
| text | #0F172A | 正文 |
| muted | #64748B | 辅助说明 |
| border | #E6EBF1 | 描边 |

## 形状与栅格

- 卡片圆角 16pt，控件圆角 11pt；
- 全部间距落在 8pt 栅格上（8 / 12 / 14 / 16 / 18 / 26）；
- 正文 13.5pt，标题 15–20pt，数字结论 20–24pt 加粗。

## 品牌标识（App 图标与界面 Logo 是同一个东西）

标识只有一个形态：**青绿对角渐变的方底座，上面一颗偏左的白色星芒**。 欢迎页、关于页、打包用的 `Lumo.icns`、README 顶部的 `docs/logo.png` 全部使用它，不允许有任何一处另起炉灶。

**三条硬性约定，每一条都是被否掉一版之后才定下来的：**

- **星芒必须是白的。**早先它是深墨绿、压在偏上的位置——整枚标识会读成 「一张脸上一只眼睛/一道疤」。深色小斑块 + 圆角方底 + 偏上，就是这个构型。 换成白色之后它从「暗块」变成「高光」，也才对得上名字——流明是光通量的单位。
- **星芒是正四角，不拉长。**拉长到过 2.00 倍，细得像一根针；1.85 / 1.65 / 1.45 都试过。 正四角星的比例本身是被设计过的，硬拉两头不讨好。
- **偏左，但纵向必须居中。**偏左让右侧的留白成为画面的一部分 （像「标记 + 预留文字位」的排版）；但横向一旦偏移，纵向就**必须**居中， 否则重心会歪——早先偏在左上角就是这个问题。



26pt



56pt



92pt

| 项 | 值 | 说明 |
| --- | --- | --- |
| 渐变起 | `#14B8A6` | 左上角 |
| 渐变止 | `#34D399` | 右下角，**变亮不变暗** |
| 形状 | `1 : 1` | 正方形底座是刻意的取舍：macOS 的图标槽位就是正方形， 底座若做成别的比例，同一个标识在桌面和窗口里就是两种形状 |
| 圆角比 | `0.26` | 圆角 / 边长。连续圆角；0.30 往上开始像胶囊，0.26 才是「一方底座」 |
| 星芒形状 | 四角星（手绘路径） | 四个外顶点指向正上 / 右 / 下 / 左，四个内凹点落在对角线上。 **不用字符 U+2726**：手绘路径才能在横向偏移这类改动里保持形状可控 |
| 星芒大小 | `0.28` | 星芒边长 / 底座边长。**高宽同值**——这个比例里没有「拉长」这个旋钮 |
| 星芒位置 | 偏左 · 纵向居中 | 左留白 0.13（× 边长）；纵向由几何撑满居中，不写死数字 |
| 星芒内凹比 | `0.30` | 内顶点 / 外顶点。0.30 比默认的 1/√2 凹得更深，四个角才显得利 |
| 星芒颜色 | `#FFFFFF` | 纯白，不是深墨绿（理由见上） |

> 以上是**唯一基准**，写在 `Sources/LumoApp/Design.swift` 的 `LumoDesign.LumoMark` 里。 `Scripts/make_icon.py` 的 `GRAD_START / GRAD_END / STAR_FILL / CORNER_RATIO / STAR_SIZE_RATIO / STAR_INSET_L / STAR_INNER_RATIO` 是照着它写的， 两边文件头互相注明。改一处就改两处——这是规范，不是建议。 同一个脚本还产出 README 顶部那个 `docs/logo.png` ， 所以它不可能是「手截的图」、也就不会在改了参数之后忘了更新。
>
> 这条规范存在的原因很具体：一旦「标识」的定义散在几处，两边迟早会各长各的—— 渐变方向可以相反、星芒可以一白一深、圆角比可以一个 0.26 一个 0.225。 写死参数就是为了让这种漂移**在纸面上不可能**。

## 交互

- 每一个需要用户判断的地方，都先给出系统推荐值；
- 能预览的一律预览：体检结论、增强前后对比、预估体积；

- 异步操作必须有进度反馈，并且进度文字说明「正在做什么」，而不是一个转圈。
- **任何一步都不能是死胡同**：选了文件之后界面上必须始终留着「换一个文件」的入口。 处理完的文档不该逼用户退出 App 才能处理下一个。

### 窗口生命周期

| 动作 | 期望行为 |
| --- | --- |
| 点窗口关闭按钮 | 窗口关闭，App 不退出 |
| 关闭后再点 Dock 图标 | **把窗口叫回来**，不是无响应 |
| Command + N | 开一个新窗口，每个窗口有自己独立的文件与处理状态 |
| Command + W | 只关当前窗口 |

> 引擎（Core Image / Vision 管线、编码器缓存）全应用共享一份， 「当前文件 / 预览 / 处理结果」每窗口一份。这两类状态混在一起会出两类 bug： 要么第二个窗口能看到第一个窗口的文件名，要么同一个文件被两个窗口并行处理时互相踩内存。

> 有一个反例值得写下来：**删掉「新建窗口」菜单项来"禁掉多窗口"**。 它的副作用是关闭窗口之后，**没有任何常规途径能把窗口叫回来**——点 Dock 图标也不响应， 只能彻底退出重开。窗口生命周期不要靠删菜单项来管理，要靠 `applicationShouldHandleReopen` 显式实现； 单窗口这件事由场景类型（ `Window` 而不是 `WindowGroup` ）保证，而不是靠"不给入口"。

## 04 · 技术架构：为什么是零依赖

Lumo 只链接系统框架：CoreGraphics 负责 PDF 渲染与手写 PDF， Core Image 负责中值滤波与非锐化掩膜，Vision 负责文字识别， ImageIO 负责 JPEG / JPEG2000 / TIFF，zlib 负责无损压缩。 用户下载下来的 DMG 解压即用，没有 Python、没有 Homebrew、没有模型文件。

这个决定不是免费的：macOS 的 SDK 没有公开的 JBIG2 编码器（见 09 节）， OCR 的语言能力由系统提供而不能像 Tesseract 那样按需装语言包。 但换来的是「装完就能用」，对一个本地工具来说，这一点值回票价。

## 05 · 内存：处理能力不该被文件大小卡住

一次要处理几百页 A4 彩色扫描件时，最占内存的从来不是 PDF 文件本身，而是 **页面的位图**。A4 在 300 dpi 下的 RGB 位图是 2480 × 3508 × 3 ≈ **26 MB**， 转成 CoreGraphics 的 32 位上下文就是 **33 MB/页**。

| 做法 | 10 页彩色文档的驻留峰值 |
| --- | --- |

| | |
|---|---|
| 位图不主动释放 + 编码结果全部攒在内存里再落盘 | 约 660 MB 起，随页数线性上涨 |
| **逐页自动释放 + 编码结果流式落盘（当前做法）** | 与「正在处理的一两页」同量级，与总页数无关 |

两处修正缺一不可：

- **及时释放**——渲染 / 增强 / 编码走的是 CoreGraphics 与 Core Image，它们产生的对象交给 autorelease 管理。在循环里不套 `autoreleasepool`，这些位图要等整个函数返回才回收，于是「处理一页释放一页」变成了「处理 N 页攒 N 份」。
- **流式落盘**——PDF 手写器改成每写完一页就把该页的字节倾倒到磁盘并清空缓冲，不再把所有编码数据攒成一个数组最后一次性写出。

> 流式写入带来一个隐蔽的记账问题：分次倾倒之后，写入器内部的「当前缓冲长度」不再是「文件中的绝对位置」，而 PDF 的交叉引用表（xref）和 `startxref` 记录的必须是**绝对偏移**。少加一个「已落盘前缀长度」，肉眼完全看不出来——PDF 照样生成、照样能打开，只有页数一多、缓冲倾倒过若干次之后，xref 才会整体指偏，表现为某些阅读器打不开或跳页。这个 bug 只有在**刻意让每页图像大小不同**的测试里才会暴露，自检断言已按这个方式构造。

流式写入还要求对象编号布局**提前定死**：PDF 的 `/Pages` 节点里要列出所有页面的 `/Kids`，而这个节点写在第一页之前，所以「第几页用哪几个对象号」在开始写之前就必须算好，不能像以前那样 根据「这份文档有没有文字层、用不用 MRC」动态决定。

## 06 · 流水线

| 阶段 | 输入 | 做的事 | 产出 |
|---|---|---|---|
| 感知 | PDF | 抽样渲染 ≤6 页，判色彩模式 / DPI / 倾斜 / 噪声 / 背景不匀，全文数文字层字符 | Analysis |
| 推荐 | Analysis | 决定增强模式、要不要 OCR、压缩路由；生成三档方案并各自预估体积 | Recommendation + 3 Plans |
| 增强 | 页面位图 | 纠偏 → 去网纹 → 模式处理 → 锐化（顺序不可换） | 增强后位图 |
| 识别 | 增强后位图 | Vision 逐行识别，写入不可见或可见文字层，附 ToUnicode CMap | 可搜索 / 可编辑 PDF |
| 压缩 | 增强后位图 | 逐页判定编码方式，必要时分层（MRC） | 最终 PDF |
| 校验 | 产物 | 重新打开、重新光栅化，逐页断言非空、非负片 | 可信的产物 |

> 两条硬规则：**一**，处理分辨率只降不升——源件只有 150 dpi 时 重采样到 200 dpi 不会变清晰，只会凭空多出 78% 的像素； **二**，每个滤镜过完都要过「内容守卫」：直方图对比度掉到原图 60% 以下就视为把内容洗掉了，放弃该滤镜。

## 07 • 增强模式

最直白的做法是把「纠偏 / 背景去除 / 去网纹 / 文本锐化」四个开关全摊在界面上。 主流手机扫描 App（扫描全能王、布丁扫描、汉王扫描）的用户研究给了一个更清楚的答案： 用户想选的是**效果**，不是滤镜。于是 Lumo 把它们收敛成五个模式：

| 模式 | 适用 | 做法 |
| --- | --- | --- |
| 自动增强 | 不确定 / 灰阶文档 | 逐页判断：文字页走黑白，含图页保色彩 |
| 黑白 | 合同、票据、纯文字 | 只留纯黑与纯白，底灰、阴影、污渍全部清掉 |
| 增强 | 带图表、印章、照片 | 保留彩色与插图，清背景、校正光照、略提饱和，**绝不二值化** |
| 对比度 | 发灰、太淡的扫描件 | 保持原有色调，按 2%/98% 分位数把明暗拉满到 0–255 |
| 原图 | 只想识别或压缩 | 不做画面处理，只做识别与压缩 |

### 为什么是 5 个，而不是照搬布丁扫描的 8 个

布丁扫描处理完照片后给出 8 个选项：**原图、增强、AI 超清、增亮、去屏纹、柔和、黑白、灰度**。 把这 8 个逐一对照 Lumo 的能力，会发现它们并不在同一条坐标轴上—— 有的在描述**输出形态**（原图 / 灰度 / 黑白），有的在描述**单点调整**（增亮 / 柔和）， 有的属于**凭外部模型猜内容**（AI 超清）。三条理由决定了取舍：

- **「灰度」与「对比度」是同一件事。**「灰度」描述的是「不要彩色」， 但 Lumo 的自动路由本来就会给纯文字页做灰度化；真正需要用户决定的 「要不要保留色彩」由「增强」这一档承担。再单独列一个「灰度」， 用户面对的就是两个会给出相似结果的按钮。
- **「AI 超清」不做，也不该做。**它的本质是用一个生成模型**猜**笔画细节， 对合同、票据这类文档是不可接受的风险——它会让「扫描件」不再忠于原件。 保持轻便是这个项目的前提：零外部模型、零网络请求、只用系统自带 SDK。
- **「增亮」「柔和」是参数，不是模式。**它们各自只改一个维度， 而 Lumo 的「对比度」可以覆盖「太淡」，「增强」已经内置了光照归一化。 把它们提成顶级模式会把「选一种效果」重新变成「填一张参数表」—— 正是这一节开头要避免的事。

结论：**对扫描版 PDF，用户真正需要区分的是「留住色彩还是只要黑白」， 再加一个「我怕软件改坏了，给我原样」的退路。**5 个模式把这条判断讲清楚了， 不需要 8 个。剩下的细粒度调整留在「微调」里，需要的人自己去找。

## 黑白档的核心算法

即局部自适应二值化，也是扫描全能王那类产品「一键黑白」背后的思路：

```
mean = 滑动窗口局部均值（窗口 ≈ 页宽 / 110, 最小 9）
out = v < mean × 0.90 ? 0 : 255
```

关键在 `× 0.90` 这一项。如果直接拿局部均值当阈值， 渐变背景（阴影、光照不匀）会在偏暗的一侧整片发黑——那一片的均值本来就更暗。 乘一个小于 1 的系数相当于给背景留出余量，而文字像素远低于均值，几乎不受影响。 窗口如果太小就不像「局部」，太大会把整页糊成一坨，所以它跟着分辨率走。

### 为什么不用 Sauvola（一次被实测否决的升级）

学术界更常见的做法是 Sauvola（ `T = μ[1 − k(1 − σ/R)]` ， 文献默认 `k=0.2~0.5、R=128` ）或 Wolf。它显然更"名门正派"， 我一度换成了 `k=0.34、R=128` ，然后在沙箱里用 numpy 把两套算法逐像素跑了一遍，结果和预期相反：

| 页面 | 局部均值×0.90 | Sauvola(k=.34,R=128) |
| --- | --- | --- |
| 高对比度合成页 | 正常 | 正常 |
| 光照渐变页（左亮右暗） | 两侧均衡 | **亮侧整片消失** |

根因是 `R` 的标定场景不同： `R=128` 是按高对比度文档定的，而真实扫描件笔画像素的标准差往往只有 10~20， 于是 `σ/R` 很小、系数掉到 0.66 附近， 浅笔画与暗侧大片被推到了阈值以上，直接判成背景。 把 `k` 压到 0.10 能救回来， 但与 `μ×0.90` **逐像素完全相同**——等于绕回原点。 Sauvola 的价值全在 `σ` 项上，而**在本项目的目标场景里，σ 项带来的不是区分度而是误判**，所以保留原来这个更简单、也更符合直觉的做法。

> 顺带记一个方法论上的坑：做这次对照时，第一版测量脚本用积分图近似滑动窗口， **边缘语义与实现不同步，内部区域就差 31 个灰阶**。据此得出的两个结论全是错的。 改用暴力真窗口做基准并交叉验证（内部误差 0.0000）后重跑，才有上面这张表。 **测量工具本身要先被验证。**这件事现在已经固化成 CI 里的回归断言。

> 二值化会把孤立噪点放大成黑斑，所以在噪声偏高时先做一次中值滤波。 同时设有退化保护：结果「几乎全黑」或「几乎全白」时退回 Otsu 全局阈值，再不行就用原图。 **已知边界**：窗口有 9 像素的下限，窄页面（宽 < 约 1000px）上的细笔画可能在 「窗口内均值 ≈ 笔画灰度」时被挖空，这是所有局部阈值法（含 Sauvola / Wolf / Niblack）的共性， 不是这一版的缺陷。

## 先看再跑：效果预览

参数描述得再清楚，也不如让用户直接看到结果。增强卡正下方就是「处理前 / 处理后」 并排预览，可翻页，切换模式或点刷新时按当前设置即时重算这一页 （只渲染一页、只跑增强，不做 OCR 不做压缩，所以几百毫秒出结果）。 放在这个位置是因为它紧挨着做决定的地方——选完模式立刻能看到，不必先跑完整流程再后悔。

> 纠偏 / 去网纹 / 文本锐化收进「微调」而不是和模式并列：它们只在少数文件上才有用，摆在模式旁边会把「选一种效果」变成「填一张参数表」。

## 08 · 压缩

### 自适应逐页路由

- **单色页**（中间调像素 < 12%）→ CCITT Group 4，文字页通常能省 90% 以上；
- **灰阶页** → **JPEG**，质量跟随用户滑杆；
- **彩色页** → **JPEG**。

### 分层压缩（MRC）

照片式扫描件——手机拍的、有阴影有噪点——用 JPEG 压整页时，为了保住文字边缘只能给高码率，于是纸纹和噪点也被高保真地存了下来。但人眼对**背景**的分辨率极不敏感（它只是一张纸），对**文字边缘**却极其敏感。把两者拆开分别编码，就是 MRC（Mixed Raster Content），也是目前扫描件压缩的事实标准：

| 层 | 内容 | 编码 | 分辨率 |
|---|---|---|---|
| 背景 | 纸、阴影、插图 | JPEG（低码率；选了 JPEG2000 时跟着走 JPX） | 1/3 |
| 文字 | 笔画 | 1bit 蒙版，无损 Flate | 全分辨率 |
| 墨色 | 实测的笔画平均色 | 填充色，随蒙版上色 | — |

> Lumo 的 MRC 带**体积闸门**：只有当「背景 + 蒙版」真的比单层编码更小时才采用。这条规则让这个特性不可能让结果变差——判断错了最多是退回已经验证过的单层路径。蒙版极性同样由 64×64 探针在真实 PDF 渲染层上实测，而不是假设。

> CI 实测（合成拍照扫描件，2 页，1654×2339@200dpi）：单层 JPEG **2,912,822 B** → 分层 MRC **78,229 B**，**省 97.3%**；产物重新光栅化后每页 std≈26.6、深色像素占比≈1.5%，即内容完好。

## 09 · 关于 JBIG2 的诚实说明

Acrobat 的单色压缩选项里有 JBIG2（有损 / 无损）。macOS 的 SDK 能**解码** PDF 里内嵌的 JBIG2（CoreGraphics 内部带了私有的解码库，预览 app 能打开这类文件就是证明），但 ImageIO **没有提供 JBIG2 编码器**——可用编码格式清单里没有它，系统里也没有公开可调用的编码入口。这一条不是推测，App 内置的 `lumo-cli encoders` 会在用户的机器上现场把 `CGImageDestinationCopyTypeIdentifiers()` 打印出来，可以自己验证。

会不会因此丢掉特色？不会，理由有三：

- JBIG2 相对 CCITT G4 的优势来自「符号字典 + 软模式匹配」， 官方数据是 3–5 倍；而扫描件真正的大头从来不在单色层， 而在**要不要为背景付出全分辨率码率**——这正是 MRC 解决的问题；
- JBIG2 有损模式有众所周知的「字符替换」风险，文档场景不可接受；
- Lumo 的差异点不在某个具体编码器，而在「感知 → 推荐 → 校验」这条自动工作流本身。

> CI 在 macOS 14 arm64 上现场体检的结果： `CGImageDestinationCopyTypeIdentifiers()` 返回 **21 种可写格式**（ `public.jpeg / public.jpeg-2000 / public.png / public.tiff / com.adobe.pdf …` ），`CGImageSourceCopyTypeIdentifiers()` 里同样没有 JBIG2。 换句话说：不是「我们没找到」，是系统确实没有公开这一层。

> 如果未来系统提供了 JBIG2 编码入口，接入只需要在单色路径里加一个分支； 在此之前，用户选择 JBIG2 时会得到 CCITT G4，并在结果里如实注明。

> 顺带回答一个常被问到的方向问题：**既然不追 Acrobat 的参数表， macOS 原生最擅长、也最主流的算法与格式是什么？** 答案是四样： 单色 **CCITT Group 4**（传真时代就在用的无损游程编码，CoreGraphics 直接支持，纯文字页压完常常只剩几十字节）、彩色与灰阶 **JPEG**、无损层 **Flate + PNG 预测器**、整页照片件 **MRC 分层**。这就是系统 SDK 的"母语"，不需要为了让参数好看而引入任何第三方编码器。

## 实测：为什么是 JPEG，以及为什么 HEIC / AVIF 用不了

ImageIO 能写 22 种格式（ `lumo-cli encoders` 会现场打印清单）， 但**能在 PDF 里用的只有那几个标准图像滤镜**：DCTDecode（JPEG）、JPXDecode（JPEG 2000）、 CCITTFaxDecode、FlateDecode、LZW、RunLength。
所以 **HEIC / AVIF / WebP / JPEG XL 一个都用不上**——系统能编码，但 PDF 里没有对应的 滤镜，写进去没人认识；JBIG2 有滤镜而 macOS 没有编码器（见 09 节）。 先确认"哪些格式合法"，再谈"哪个更小"，顺序反了会做很多无用功。

| 编码 | 彩色页（示例件 03） | 灰阶页（示例件 02） | 单色页（示例件 01） |
|---|---|---|---|
| JPEG | **356 KB** · PSNR 50.5 | **424 KB** · PSNR 51.4 | — |
| JPEG 2000 | 720 KB · PSNR 50.4 | 720 KB · PSNR 49.3 | — |
| CCITT G4 | — | — | **14 KB** · 无损 |
| Flate（无损） | 2.12 MB | 1.30 MB | 20 KB |

读法：**比 PSNR 相近的那两行**——同样保真度下谁的字节少，谁才是更好的编码器。 彩色页在 50.5 dB 时 JPEG 356 KB、JPEG 2000 720 KB，差了一倍；灰阶页同样是 1.7 倍。 所以默认值全部落在 JPEG 与 CCITT G4 上。复现： `lumo-cli encbench <扫描件> --page 3 --quality 60,70,80,90` 。

### 无损这一档：为归档加的一个选项

三档预设之外，彩色 / 灰阶编码里还有一个**无损 JPEG 2000**。 它和「ZIP 无损」是同一类（都不量化），但实测在彩色页上**体积只有一半** （示例件第 3 页 1.05 MB : 2.12 MB），而且编解码之后**逐像素相同**—— 这条是自检里的一条断言（把编码结果解回来与原图比对），不是凭说的。

选它还会顺手关掉两件事，否则"无损"就只对最后一步成立： **不做自动二值化**（二值化本身就是丢像素）、**不走 MRC 分层** （分层的背景层是有损 JPEG）。第二条以前连 ZIP 都没做到——勾着自适应选 ZIP， 页面还是会被 MRC 拿走。

### 为什么是 JPEG，而不是"更新更好"的 JPEG 2000

常听到的说法是「JPEG 2000 压缩比更高还能保持质量」。这在**好的 JP2 编码器 + 典型照片内容**上确实成立（小波没有 8×8 块效应，低码率下更干净）。 但我们要比的是**系统 ImageIO 里那个 JP2 编码器**，实测它在这份材料上并不占优：

| 码率（bits/px） | JPEG PSNR / SSIM | JPEG 2000 PSNR / SSIM |
|---|---|---|
| 约 1.0 | 48.6 dB / 0.994 | — |
| 约 1.7 | **53.2 dB / 0.998** | 45.3 dB / 0.980 |
| 约 2.7 | — | 50.4 dB / 0.994 |

同样 1.7 bits/px 时，JPEG 在**两个指标上都明显更好**——所以这不是 PSNR 的偏袒。 另一个旁证：把 JP2 的质量旋钮从 80 拧到 90，**产出的字节数完全一样**， 说明那个旋钮在高段已经饱和了。再加上一个方法学上的坑： 我们的示例页本身是 JPEG 压过的，重编码成 JPEG 相当于"顺着原有的artifact 走"， 换成 JP2 反而要重新建模——所以这个结论**只对这份材料 + 这台机器上的编码器成立**， 换了编码器要重测（`lumo-cli encbench` 就是为这件事准备的）。

结论：默认走 JPEG，JPEG 2000 作为选项保留（要无损时它反而是最优的那一档）。

> 同一个实验还纠了一个**记错了很久的结论**。代码里原本写着 「PDF 的 Flate 预测器不能带行首 filter 字节，加了会整页空白」。
> 一次带五种写法的交叉验证（ `lumo-cli selftest` 的 「Flate 与 /Predictor 交叉验证」）把它推翻了：**带字节**的 Predictor 12 / 15 解回来与原图**逐像素相同**（平均差 0.00），**不带字节**的那版才是乱码（差 123）， 而当年被认为"能用"的 TIFF 预测器（Predictor 2）其实是错的（差 109）。
> 当年看不出来，是因为判据用的是「对比度」——那个判据连错的写法都放过去了（它拿了 190 分）。 现在的判据是**逐像素差**：能区分"看着有内容"和"真的解对了"。
> 结果是 8 位路径改用 **/Predictor 15**（逐行自选滤波方式），无损体积再降一成左右； 1 位路径仍然不用——PNG 滤波作用在字节上，而 1 位是一字节装 8 个像素， 「/Columns 按采样还是按字节」各家阅读器做法不一致，不值得赌（单色本来由 G4 罩着）。

# 10 · 已验证与已知限制

### 已验证（在真机 macOS arm64 上断言过）

- 体检各项指标、纠偏角度往返一致性；
- 全流程产物重新打开、重新光栅化，逐页非空、非负片；
- CCITT G4 / JPEG / JPEG2000 / ZIP（含 /Predictor 15 预测器）/ 分层压缩五条编码路径； 另有一条交叉验证：五种 /Predictor 写法各写一份 PDF 渲染回来**逐像素比对**， 确认"带行首 filter 字节"才是对的；
- 可搜索与可编辑两种 OCR 输出、页码范围选择、产物再体检能认出自己的文字层；
- 自动裁边的四道闸门与透视校正——几何层用**显式构造的四边形**驱动，不与模型耦合；
- 更新器：版本比较、校验（哈希 / 签名 / 缺签名拒绝）、原地替换与失败回滚、下载全链路； 另有一条端到端用例：起一个假的更新源，真的从旧版本升到新版本。

### 已知限制

- OCR 语言由系统提供，语言清单随 macOS 版本变化；
- 没有 JBIG2 编码（原因见 09 节）；
- 分层压缩只对「非单色、含文字」的页面生效，纯图片页仍走 JPEG/JPEG2000；
- Flate（ZIP 无损）不使用 PNG 预测器：PDF 的 /Predictor 与 PNG 文件格式语义不同， 同一份像素实测「无预测器 215 / Predictor 12: 41 / Predictor 2: 190」（对比度，越高越正确）， 因此选择不用。代价是 ZIP 模式体积比理论值大 20%~40%，换来「一定能打开」；
- 不支持 PDF 加密文件与表单域的改写。

# 11 · 设计取舍记录

这一节只记一类东西：**看起来可以随便选、其实被实测否掉过别的选项**的地方。 结论容易抄，理由抄不走——而只看到结论的人，会理所当然地把它「优化」回那个已经被证伪的选项。 这就是本节存在的唯一理由。

### 0.1.0

| 项 | 取舍 | 依据 |
| --- | --- | --- |
| 倾斜的汇总量 | 体检报告里显示**最歪的那一页**，而不是各页的**平均值** | 三页分别是 +0.9° / −1.5° / +0.5° 时，平均值是 −0.03°—— 界面上显示"倾斜 0.03°"，看起来毫无问题，而第 1 页肉眼就是歪的。 更糟的是"要不要纠偏"也吃这个平均值，于是**永远不推荐纠偏**。 倾斜本来就是逐页的性质，取平均在语义上就是错的 |
|  | 从 1.0° 下调到 **0.3°** | 0.3° 在 A4 页面上对应横向 1.3 mm 的落差，几行文字就看得出来。 原来的 1.0° 让"看得出歪"的页面被放过（示例件第 1 页 0.9°） |

| | | |
|---|---|---|
| 纠偏的阈值 | | |
| 默认处理分辨率 | 跟随**源件**的 dpi（钳在 150~300），不再写死 300 | 写死 300 的后果实测过：150 dpi 的扫描件被渲染到 2480×3506（4 倍像素）， 输出比源文件还大 6.5%（808 KB vs 759 KB），而界面预估只有 77 KB。 项目自己的规则写着"只降不升"，这条默认值是漏网的 |
| 无损这一档 | 新增**无损 JPEG 2000**；并且选了无损就**不走 MRC、不自动二值化** | 无损 JP2 在彩色页上比无损 Flate 小一半且逐像素相同。 而 MRC 与二值化都会丢像素——留着它们，"无损"就只对最后一步成立 |
| 对比方式 | 并排之外增加**窗帘**（单页、拖动竖线比同一处） | 并排看的是"两张图"，靠眼睛来回扫；而"这一处到底改成了什么样"，只有把两层叠在同一个位置才看得准——尤其是渐变背景与很细的文字边缘 |

## 0.0.2

| 项 | 取舍 | 依据 |
|---|---|---|
| 标识的形状与构图 | **方底座**（1 : 1）+ 左上角一束**纵向拉长**的星芒。试过竖版底座，否掉了 | 竖版底座的问题不在好看与否，而在**它和图标不是同一个形状**： macOS 的图标槽位是正方形，于是同一个标识在桌面和窗口里长得不一样， 又要多写一段"这是平台约束、不算走样"来解释。方形底座让图标、欢迎页、 关于页、README 的 logo 是同一份几何，这个解释成本直接消失。星芒偏到左上、并纵向拉长，则是因为：正四角星居中会读成一个「点」， 没有方向；偏置加上拉长之后，右下的留白才成为画面的一部分 |
| 顶部不再有品牌 | 连标识带「Lumo」字样一起**从窗口顶部移走**，只留在欢迎页中央 | 顶部放一个偏置构图的标识最尴尬：它小到看不清构图，却仍然占着一行高度。 一个 App 有一处大标识就够了 |
| 欢迎页的虚线框 | 去掉那圈**常驻**的虚线框，改成"文件真的拖进来时才亮一下" | 常驻的框会让欢迎页读成一个空表单，而不是"随时可以放东西进来"的画面。 反馈应当只在需要时出现 |
| 发布说明的来源 | Release 正文**从 CHANGELOG 抽**，不用 GitHub 的自动生成 | 自动生成只列合并的 PR——实测出现过整篇发布说明只有一条机器人依赖升级、 真正的改动一条都没有。发布说明是给下载的人看的，它必须来自一处 按「用户能感知的变化」写的地方 |
| 顶部那一栏 | **整条去掉**：不留材质底、不留分隔线，只留一个标识 + 一个词标 | 去掉之后才看出来，原来那一栏的视觉重量有相当一部分来自那条 1px 分隔线。 顶栏本来还承担「放设置按钮」的职责，那个职责挪到右下角更 |

| | | |
|---|---|---|
| | | 合理—— 右下角是「全局动作」的常规位置，而顶栏那块地方应该留给内容 |
| 空态文案 | 删掉两行：流水线四段式的一句话、以及一排快捷键提示 | 「感知 → 增强 → 识别 → 压缩」是**内部**的说法，用户在那个时刻并不需要它； 快捷键在菜单里都有，堆在首屏只会让「该点哪个」变得不清楚 |
| 界面语言 | 中 / 英双语，默认跟随系统 | 英文界面用一张「中文原文 → 英文」的表，不做 .lproj： 本项目的 .app 是手工组装的，没有 SwiftPM 资源包那条链路， 引 .lproj 要动打包脚本、还要多一条「资源有没有拷进包里」的断言。 中文侧不维护副本（查不到就原样返回），英文侧漏翻由 CI 拦住 |

## 0.0.1 · 首个公开版本

| 项 | 取舍 | 依据 |
|---|---|---|
| 增强模式 | 收敛成 **5 个模式**（自动增强 / 增强 / 黑白 / 对比度 / 原图）， 不照搬同类产品的 8 个 | 用户想选的是**效果**，不是滤镜。模式名直接说结果—— 「彩色增强」其实在说"你要不要彩色"，「黑白增强」的"增强"二字是多余的 |
| 黑白档的阈值算法 | 局部均值 × 0.90。**不用**学术上更常见的 Sauvola | 逐像素对照过两套算法：Sauvola(k=0.34, R=128) 在光照渐变页上把亮侧整片抹白， 根因是 R=128 按高对比度文档标定、与真实扫描件的 σ 量纲不符； 把 k 压到 0.10 能救，但**退化成与局部均值算法逐像素相同**。 ——「更学术」不构成换算法的理由，实测才构成 |
| 单色编码 | CCITT G4（单色） / JPEG2000（彩色） / Flate（无损）—— 直接落在系统 SDK 的「母语」上 | JBIG2 的压缩率更好，但它**没有公开编码器**（实测，不是推测）。 为参数好看去引一个第三方编码器，与本项目「零第三方依赖」的前提直接冲突 |
| 处理分辨率 | 只降不升 | 源件只有 150 dpi 时，重采样到 200 dpi 不会变清晰， 只会凭空多出 78% 的像素 |
| 内容守卫 | 每个滤镜过完都检查一次直方图对比度，掉到原图 60% 以下就放弃该滤镜 | 增强滤镜的失败方式是「画面变干净了、内容也没了」。这个方向不可逆， 所以宁可留着脏，也不能把内容洗掉 |
| 内存 | 手写流式 PDF 写入器，逐页落盘 | 把编码数据攒齐再写的话，几 MB 的文档处理时内存会涨到几个 GB—— 页数不应该是内存的函数 |
| 自动裁边 | 默认**关闭** | 它是整条流水线里唯一会主动丢像素的一步。默认关， 把是否丢像素的决定留给使用的人 |

| | | |
|---|---|---|
| 兼容性 | 在 macOS 14 与 26 两代系统上都验证 | 低版代表支持下限（`LSMinimumSystemVersion` = 13），新版代表工具链上限。只在一个系统上绿，说明不了另一个——实测过：同一份代码在新系统上零警告，在旧系统上会编不过 |

> 这条纪律值得单独记一笔：**断言本身也要被测试**。写完一条断言，要先用「故意写错的产物」跑一遍，确认它**会红**——否则你得到的不是保障，而是一行每次都会打印出来的、绿色的安慰语。本项目里出现过三种形态的假断言：恒真的往返比对、范围写错导致一个条目都没扫到、以及用全局均值去判一个只影响 1% 画面的异常。

© Lumo · 本文档由 Lumo 项目生成，随 App 内置发布。